

DATA STRUCTURES

Handwritten Notes

VIVEKANAND VERNEKAR

GateWallah Parakram C 2024

(Hinglish Weekday)

Other Notes Uploaded Below

<https://t.me/pwgateenotes>

DATA STRUCTURES

25/11/23

Introduction:

Data Structure

Linear

1. Arrays
2. Linked list
3. Stacks
4. Queues.

Atmost 2
neighbours for each
Element.

Non linear

1. Tree
2. Graphs.

It is possible that an
element can have
more than 2 neighbours.

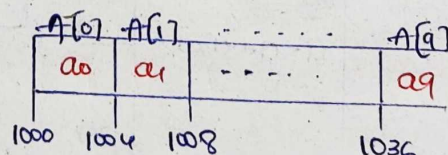
Arrays

- * Collection of similar type of elements
- * Index starts from 0.
- * Elements are stored one after another
- * Relative addressing.
- * Random access
- * Cache friendly

⇒ int → 4 byte

⇒ b.A → 1000.

int A[10];



Address of A[0] = 1000

Address of A[2] = ?

① Elements already found before $A[2] = ?$

$$\rightarrow \text{add}(A[2]) = 1000 + 8 = 1008$$

∴ So 2 elements found before $A[2]$

② Size of each element = 4 byte

③ Memory already found before $A[2] = A[2] = 2 \times 4 = 8 \text{ Bytes}$

④ Elements found before $A[6] = \text{index } 0 \text{ to } 5 = 5 - 0 + 1 = 6$
& Memory found = $6 \times 4 = 24 \text{ Bytes}$

→ Theoretically Index can start from any number

$$\begin{aligned} \text{Arr}[-5 \dots 5] \quad \# \text{elements} &= \text{last} - \text{first} + 1 \\ &= 5 - (-5) + 1 \\ &= 11 \text{ elements} \end{aligned}$$

lower index larger index

eg: $A[-5 \dots 5]$, $w = 4 \text{ Byte}$
Base address = 1000

How many elements found before $A[1] = \text{index } -5 \text{ to } 0$
= $0 - (-5) + 1 = 6$
& Memory found = $6 \times 4 = 24 \text{ Bytes}$

2-D Array

`int a[2][3];`

Every row the number of elements is equal to no. of columns

	0	1	2	
0	a_{00}	a_{01}	a_{02}	row with index 0
1	a_{10}	a_{11}	a_{12}	row with index 1

2D-Array

- Row wise stored (Row major order)
- Column wise stored (Column major order)

eg: $\text{int } a[3][4]$:

→ Rows already filled before with index 2 = 2 rows

→ No. of elements already filled before a_{23} in row with index 2 = 3 elements

→ Elements already filled before $a_{23} = 2 \times 4 + 3 = 11$

(and memory filled = 11×4

= 44 Bytes

= $1000 + 44 = 1044$ (address)

eg: $w = 2 \text{ byte}$, $BA = 1000$; $a[-5..5][-3..3]$;

$11 \times 7 = 77$ elements total

→ Elements filled before $a[i][j] =$

(rows already filled = $0 - (-5) + 1 = 6 \times 7 = 42$

+ elements in current row = 4

46 elements

→ Memory filled before $a[i][j]$

= $46 \times 2 = 92$ Bytes

Q1) Given 2D array with Row major order: $a[-5..5][-7..7]$

$w = 4 \text{ bytes}$

$BA = 1000$

$7 - (-7) + 1 = 15$
 $5 - (-5) + 1 = 11$

Elements filled before $a[i][j] =$

$\downarrow$ $-7 \text{ to } 2 = 2 - (-7) + 1$
= 10 elements

= $0 - (-5) + 1 = 6 \times 15$
= 90

∴ $90 + 10 = 100$ Elements

Memory filled before $a[i][j] = 100 \times 4 = 400 \text{ Byte}$

$1000 + 400 = 1400$ Address

Generalised formula: $a[m][n]$

address ($a[i][j]$)

rows already fixed
before row with index

$$i = 0 \text{ to } i-1$$

= i rows



$i \times n$ elements

within i^{th} row, elements

already fixed before $a[i][j]$

= cols with index 0 to $j-1$

$$= j-1-0+1$$

= j elements

Total elements already fixed = $(i \times n + j)$

Size of each = 'w' bytes

→ Memory already fixed = $(i \times n + j) \times w$ Bytes

→ Address of $a[i][j]$ = $BA + (i \times n + j) \times w$

3-D Arrays

→ `int a[3][4][5];`

Every index in
this dimension rep
resent "4x5" elements

Every index in this
dimension represent "5"
elements

eg. P_{no} : $w=4$ bytes, $BA=1000$ `a[3][3][5];`

add (a_{223})

Index already
fixed = 0 to 1

$$= 1-0+1$$

$$= 2$$

↓
 $2 \times 3 \times 5$

Index fixed

$$= 0 \text{ to } 1$$

$$= 1-0+1$$

$$= 2$$

↓
 2×5

Index

$$= 0 \text{ to } 2$$

$$= 2-0+1$$

$$= 3$$

Total elements = $(2 \times 3 \times 5)$

$$+ (2 \times 5) + 3$$

$$= 43 \text{ elements}$$

Total memory fixed

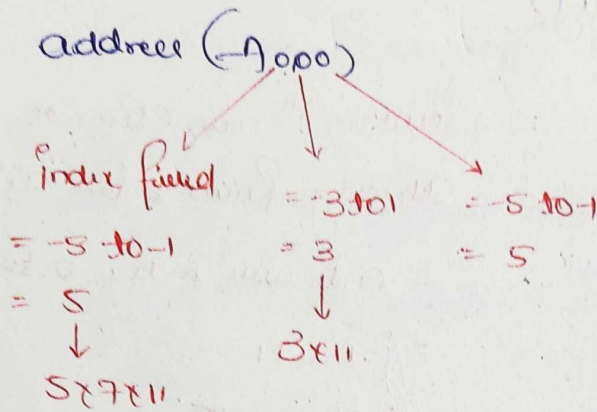
$$= 43 \times 4 = 172 \text{ bytes}$$

$$= 1000 + 172$$

$$= 1072 \text{ (address)}$$

eg 2: $w = 2 \text{ byte}$
 $BA = 1000$

$A[-5..5] [-3..3] [-5..5]$



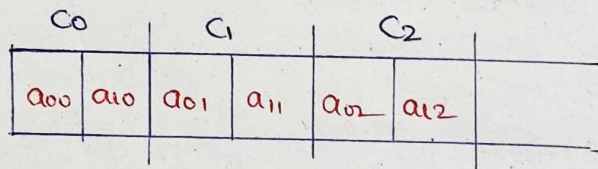
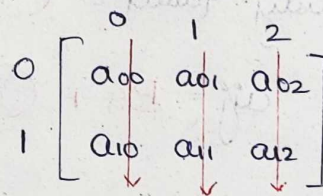
Total Elements already found
 $= (5 \times 7 \times 11) + (3 \times 11) + 5$
 $= 423 \text{ elements}$

Memory already found
 $= 423 \times 2 = 846 \text{ Bytes}$

$\Rightarrow 1000 + 846$
 $= 1846 \text{ (address)}$

2-D Array - Cmo

$\rightarrow \text{int } a[2][3]$



eg: $\text{int } a[3][4] \text{ Cmo}$

add(a_{23})

Elements found
 $= 0 \text{ to } 1$
 $= 1 - 0 + 1$
 $= 2$

Elements found
 $= 0 \text{ to } 2$
 $= 2 - 0 + 1 = 3 \rightarrow 3 \times 3$

$\rightarrow \text{Total elements} = (3 \times 3) + 2$
 $= 11 \text{ elements}$

$\rightarrow \text{Memory already found}$
 $= 11 \times 4 = 44 \text{ bytes}$

Address = 1044

eg 2: $w = 2 \text{ byte}$
 $BA = 1000$

$a[-5..6] [-3..3]$

address (A_{000})

$= -5 \text{ to } 1$
 $= 5$

$= -3 \text{ to } -1$
 $= 3$
 $\downarrow$
 3×12

$\rightarrow \text{Total Elements} = 3 \times 12 + 5 = 41 \text{ elements}$

$\rightarrow \text{Memory found} = 41 \times 2 = 82 \text{ Bytes}$

$\rightarrow \text{Address} = 1082$

Rmo $\rightarrow$

$[m][n][p]$

every index
= mnp

every index
has p elements

Cmo $\leftarrow$

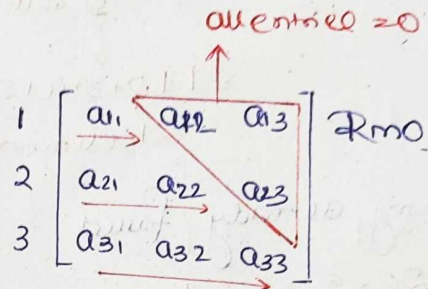
$[m][n][p]$

every index
= m

every index = n/m

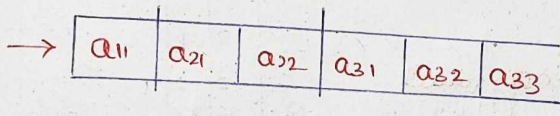
Lower Triangular Matrix

* A LTM is a square matrix
where $A_{ij} = 0, i < j$.



a_{12}
 a_{13}
 a_{23} } all zero (0)
row column

memory only in
non-zero entries



eg: $a[5][5] \rightarrow$ LTM

add(a_{43})

rows already filled
before row with index 4
= 1 to 3
= 3 rows
= $1+2+3 = 6$ elements

within row

every index 4
elements filled
before a_{43}
= col with ind. 1 to 2
= 2

$\rightarrow$ Total "elements already
filled before a_{43}

Generalised formula: add(A_{ij}) LTM $\Rightarrow N \times N$ {Rmo}

row already
filled = 1 to $i-1$
= index 1, 2, 3, ..., $i-1$
= $1+2+3+\dots+(i-1)$

= $\frac{(i-1)(i)}{2}$ elements

within i 's row
elements already filled
= col with index 1 to $j-1$
= $(j-1)$ elements

$$\text{Total elements already found} = \left[\frac{i(i-1)}{2} + (j-1) \right]$$

eg: LTM (RMO) $A[-5, 5][-7, 3]$

$w = 2 \text{ byte}$

$BA = 1000$

$\text{add}(A_{1-2})$

rows already found: -5 to 0

$= 6 \text{ rows}$

$= 1+2+3+4+5+6$

$= 21 \text{ elements}$

within row index 1, columns already found $= -7$ to -3

$= -3 - (-7) + 1$

$= 5 \text{ elements}$

$\rightarrow \text{Total elements already found} = \underline{26}$

$\rightarrow \text{Memory already found}$

$= 26 \times 2 = \underline{52 \text{ Bytes}}$

$\rightarrow \text{Add}(A_{1-2}) = 1052$

eg: LTM $A[5][5] \{C_{MO}\}$

$\text{add}(a_{53})$

$\rightarrow \text{col already found}$

$= \text{ind } 1 \text{ to } 2$

$= \text{ind } 1, 2$

$= 5 + 4 = 9 \text{ elements}$

within intended

col (3rd col)

$= (5-3)$

$= 2 \text{ elements}$

C_{MO}

1				
2	1			
3	2	1		
4	3	2	2	
5	4	3	3	1

Generalised formula: $N \times N$
(C_{MO})

$\text{add}(a_{ij})$

$\rightarrow \text{Col already found}$

index $\{2, 3, \dots, j-1\}$

within col index

each element already

found before a_{ij}

$(i-j)$

index 1 $= N - (i-1)$

index 2 $= N - (i-2)$

index $j-1 = N - (i-j+2)$

$= \frac{j-1}{2} (N + (N - j + 2))$

Total elements

$= \frac{j-1}{2} (N + (N - j + 2)) + (i-j)$

$= \frac{(j-1)}{2} (2N - (j-2)) + (i-j)$

$= N(j-1) + \frac{(j-1)(j-2)}{2} + (i-j)$

Q1) LTM (Cmo) $\rightarrow A[-3..3][-5..1]$ (7x7)

$w = 1$ byte

$BA = 1000$

add($A[3][1]$)

Already found

$$= 3 - (-1)$$

$$= 4$$

$$\Rightarrow i-j$$

$$4 - 2 = 2$$

Col already found

$$= -5 \text{ to } -2$$

$$= -2 - (-5) + 1$$

$$= 4$$

$$7 + 6 + 5 + 4 = 22$$

$$\rightarrow \text{Total Elements} = 22 + 2 \Rightarrow \underline{24}$$

$$\rightarrow \text{Memory} = 24 \text{ Byte}$$

$$\rightarrow \text{Address} = 1024$$

Upper Triangular Matrix

* It is a square matrix with

$$a_{ij} = 0, i > j$$

$$\begin{matrix} & 1 & 2 & 3 \\ \begin{matrix} 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ 0 & a_{22} & a_{23} \\ 0 & 0 & a_{33} \end{bmatrix} \end{matrix}$$

All entries are zero here

eg: Rmo - UTM $a[5][5]$

add(a_{34})

Rows already found = 1 to 2

$$= 2 \text{ rows}$$

$$= \underline{5 + 4 \text{ elements} = 9}$$

within row whose

index is 3,

$$\text{Elements found} = 4 - 3 = \underline{1}$$

$$\begin{matrix} 1 & 2 & 3 \\ 1 & 2 \\ 1 \end{matrix}$$

Rmo

$$\text{Total Elements} = 10$$

Generalised :- add(a_{ij})

Rows already found

$$1^{\text{st}} \rightarrow N$$

$$2^{\text{nd}} \rightarrow N-1$$

$$\vdots$$

$$i-1 \rightarrow N-(i-2)$$

$$= \frac{i-1}{2} [N + (N-i+2)]$$

within its row

$$\text{Elements found} = (j-i)$$

Total Elements =

$$= \frac{i-1}{2} [2N - (i-2)] + (j-i)$$

$$= (i-1)N - \frac{(i-1)(i-2)}{2} + (j-i)$$

eg: $A[-12..12][-12..12]$

$w = 2$ byte

$Bn = 1000$

add (A_{03})

rows already
found = -12 to -1
= 12 rows

$$\Rightarrow \frac{12 \times 6}{2} (25 + 14) = 234$$

elements found within row
index 0 before A_{03}
= $3 - 0 = 3$

$$\rightarrow \text{Total Elements} = 234 + 3 = \underline{237}$$

$$\rightarrow \text{Memory found} = 237 \times 2 = \underline{474 \text{ Byte}}$$

$$\rightarrow \text{Address} = 1000 + 474 = \underline{1474}$$

Utm in Cmo

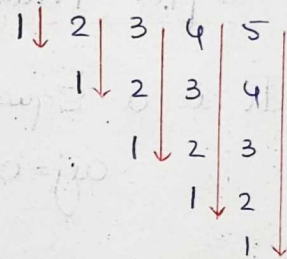
eg: $a[5][5];$

add (a_{45})

within cols
elements already found
before a_{45}
= 1 to 3
= $3 - 1 + 1 = \underline{3}$

Col already
found
= $\{1, 2, 3, 4\} \rightarrow 4$
= $1 + 2 + 3 + 4 = \underline{10}$

$$\rightarrow \text{Total Elements} = 10 + 3 = \underline{13}$$



Tri-diagonal Matrix

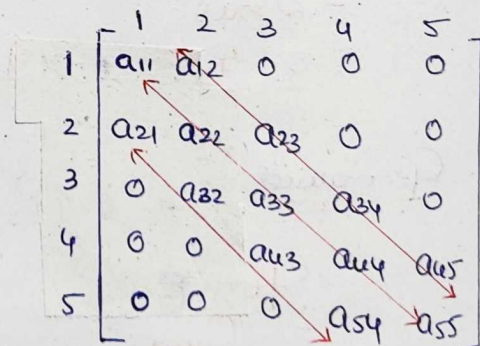
* It is a square matrix

* Main diagonal

* Diagonal just above
main diagonal

* Diagonal just below
main diagonal

except them
all entries
are
zero.



$n \times n$ Tri diagonal Matrix

Total rows = n

1st = 2

last = 2

Remaining rows = $(n-2)$

Elements in row except first & last = 3.

Total Elements = $2 + 2 + (n-2)3$

$$= 2 + 2 + 3n - 6$$

$$= 3n - 2$$

eg: Rmo - Tri diagonal matrix

add(a_{45})

rows already

fixed = 3

1st — 2

2nd — 3

3rd — 3

8 Elements

within i^{th} row
Elements before

$$a_{45} = (5-4) + 1$$

= 2 elements

a_{11} a_{12}

a_{21} a_{22} a_{23}

a_{32} a_{33} a_{34}

a_{43} a_{44} a_{45}

a_{54} a_{55}

Generalised Formula : $N \times N$ - Tri-diagonal.

Rmo

add(a_{ij})

within i^{th} row

Rows already

fixed = 1 to $i-1$

= $(i-1)$ rows

Elements already

fixed by a_{ij}

= $(j-i+1)$

Out of these $(i-1)$ rows

1st row = 2

rem. $(i-2)$ rows = 3

$$2 + 3(i-2)$$

$$= 2 + 3i - 6$$

$$= \underline{3i - 4}$$

Total Elements before a_{ij}

$$\Rightarrow 3i - 4 + j - i + 1$$

$$= \underline{2i + j - 3}$$

Disadvantages of array :

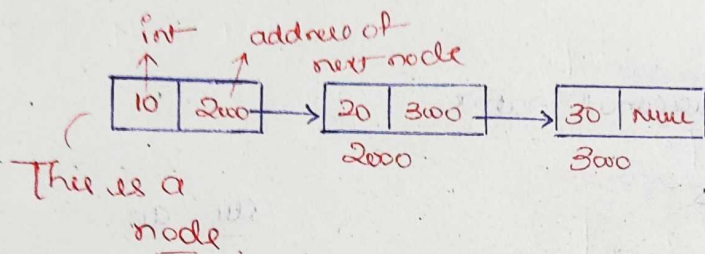
* pre allocated size

* Insertion/deletion are expensive

Linked list is a linear data structure which is a collection of elements called nodes, in which every node contains 2

parts: (I) data

(II) address of next node



```
Struct Node {
```

```
    int data;
```

```
    Struct Node *next;
};
```

Creating a node: `Struct Node *ptr;`

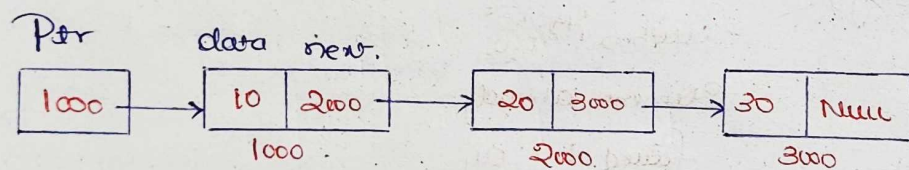
`ptr = malloc (sizeof (Struct Node));`

→ `ptr`: pointer to structure variable.

Accessing member functions:

`(*ptr).data` or `ptr → data`

`(*ptr).next` or `ptr → next`

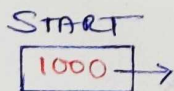


`ptr → data = 10`

`ptr → next = 200` (address of next node)

`(ptr → next) → data = 20`

↓
pointer to 2nd node



To access next node: `ptr = ptr → next;`

* `Struct Node *START;` → pointer to first node, contains only address to struct node.

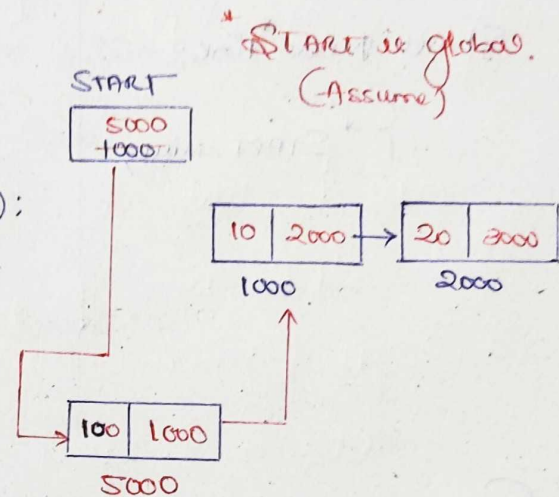
Empty Linked list

* There is no 1st node

* START cannot have any valid address i.e. $START = NULL$.

Insertion:

```
void insert_at_begin (int key)
{
    struct Node *ptr;
    ptr = malloc (sizeof (struct Node));
    if (ptr != NULL)
    {
        ptr -> data = key;
        ptr -> next = START;
        START = ptr;
    }
}
```



void insert_at_end (int key)

```
{
    struct Node *temp, *ptr;
    temp = malloc (sizeof (struct Node)); → Create node
    if (temp) {
        temp -> data = key;
        temp -> next = NULL;

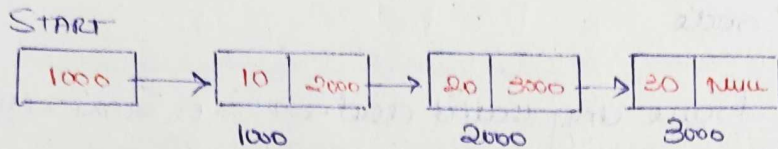
        if (START == NULL) → if linked list is empty
        {
            START = temp;
            return;
        }
    }
}
```

next ← ptr = START; → at least 1 node exists.

```
while (ptr -> next != NULL)
{
    ptr = ptr -> next;
    ptr -> next = temp;
}
}
```

} Traverse till last node
(→ add temp to last.)

Traversal



```
Struct Node
{
    int data;
    Struct Node *link;
} *START = NULL;
```

```
void Traversal()
{
    Struct Node *ptr;
    ptr = START; → START is global
    while (ptr != NULL)
    {
        printf("%d", ptr->data);
        ptr = ptr->link;
    }
}
```

Count nodes in a linked list

```
int count_node()
{
    int count = 0;
    Struct Node *ptr;
    ptr = START;
    while (ptr != NULL)
    {
        count++; ptr = ptr->link;
    }
    return count;
}
```

Print last node data :

```
void Print_last()
{
    Struct Node *ptr = START;
    if (START == NULL) return; → if no node exists
    while (ptr->link != NULL)
    {
        ptr = ptr->link;
    }
    printf("%d", ptr->data);
}
```

Print Second last node data

```
void Print_Second_last()
{
    if (START == NULL || START->link == NULL) return;
    ptr = START;
    while (ptr->link->link != NULL)
        ptr = ptr->link;
    printf("%d", ptr->data);
}
```

(Ensure that atleast 2 nodes exist)

Find whether key is present in linked list

```
void Search(int key)
{
    struct Node *ptr = START;
    while (ptr != NULL)
    {
        if (ptr->data == key)
        {
            printf("yes"); return;
        }
        ptr = ptr->link;
    }
}
```

Print alternate node data

```
{
    struct Node *ptr = START;
    while (ptr != NULL && ptr->link != NULL)
    {
        printf("%d", ptr->data);
        ptr = ptr->link->link;
    }
    if (ptr == NULL) return;
    if (ptr->link == NULL) printf("%d", ptr->data);
}
```


Deletion in linked list

27/11/2023

void delete_from_begin()

```
{ if (START == NULL) return;
  ptr = START; check if list empty
```

```
  START = START → next;
```

```
deallocate Space. free (ptr); }
```

void delete_from_end()

```
{ struct Node *ptr = START;
```

```
  if (START == NULL) return;
```

```
  else if (START → next == NULL)
```

```
  { START = NULL;
```

```
    free (ptr);
```

```
    return; }
```

```
  while (ptr → next → next != NULL)
```

```
    ptr = ptr → next;
```

```
  free (ptr → next);
```

```
  ptr → next = NULL;
```

```
}
```

Print node data in reverse

void print (struct Node *ptr)

```
{ if (ptr != NULL)
```

```
  { print (ptr → next);
```

```
    printf ("%d", ptr → data); }
```

```
}
```

→ recursively go till end then print all data.

Reverse a linked list

void reverse()

```
{
    struct Node * current = START;
    struct Node * previous = NULL, * next = NULL;
    while (current != NULL)
    {
        next = current → next; → Store next
        current → next = previous; → reverse current node's
                                   pointer
        previous = current;
        current = next; } → move pointers one position
                           ahead
    START ← head = previous;
}
```

Detect loop in a linked list

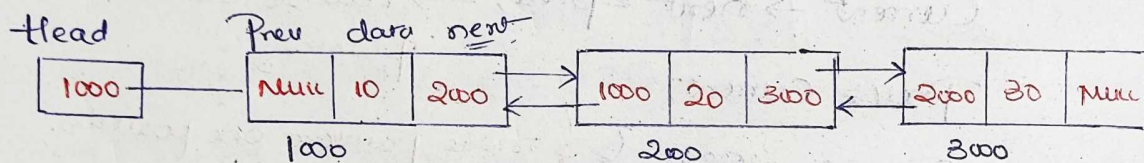
```
{
    struct Node * slow = head, * fast = head; → fast & slow
                                                pointers
    if (head == NULL) return 0;
    while (fast != NULL && fast → next != NULL)
    {
        slow = slow → next;
        fast = fast → next → next;
        if (slow == fast) return true;
    }
    return false;
}
```

↳ if cycle exists then they will become equal. Otherwise fast will reach the end if no cycle exist.

Type of linked list:

1. Singly linked list
2. Doubly linked list
3. Circular linked list
4. Header linked list.

Doubly linked list:



Struct Node {

Struct Node *prev;

int data;

Struct Node *next;

}

Insertion : { struct Node *temp = malloc(sizeof(struct Node));

temp → data = key;

temp → next = head;

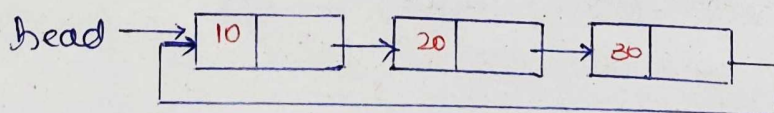
temp → prev = Null;

head → prev = temp;

head = temp;

}

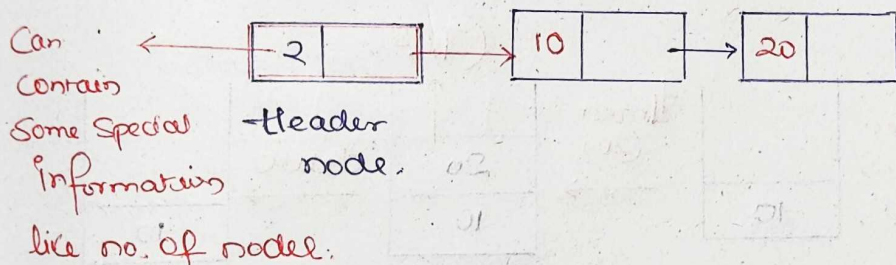
Circular linked list



Traversal in Circular Linked list

```
{ if (head == null) return;  
do { printf("%d", ptr->data);  
    ptr = ptr->next; }  
while (ptr != head);  
}
```

Header Linked list



Data Structure

Abstract view

- * No implementation
- * No programming language
- * Only about feature/operations defined

Concrete view

- * Implementation
- * Programming language

Stacks And Queue

27/11/23

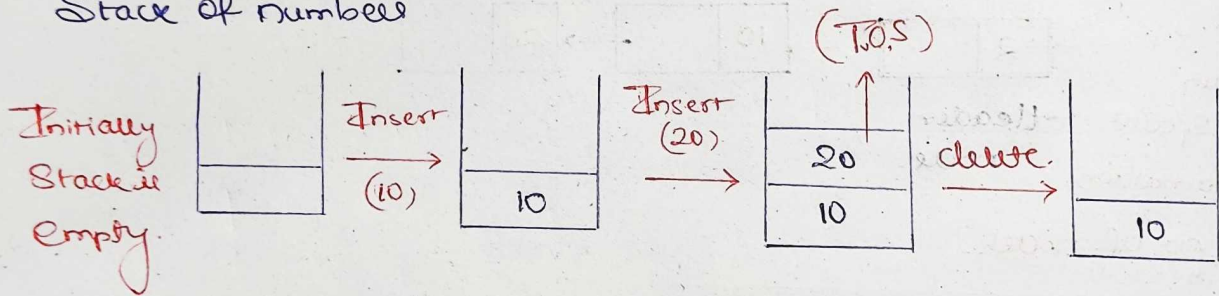
Stack:

- * Linear data structure
- * Works on last in first out policy (LIFO)
- * Order of deletion: reverse order of insertion
- * Both insertion and deletion are performed only at one end
- * Called as top of stack.

Stack as ADT:

Top: most recently added element.

* Stack of numbers



- Stack operations:
- * Insert $\rightarrow$ push
 - * delete $\rightarrow$ pop
 - * is Empty(): True if stack is empty else false.
 - * is Full():

Applications of Stack:

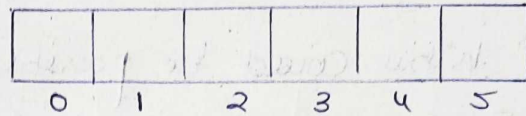
1. Recursion/function
2. Infix to prefix
3. Prefix evaluation
4. Infix to postfix
5. Postfix evaluation

Main/Sole purpose of stack
is: - wait 721011
- To delay.

Implement Stack using array:

#define SIZE 6

int STACK[SIZE];



Top: represent index of most recently added element.

* Initially, TOP = -1.

→ void Push(int x)

{ if (TOP == SIZE - 1) — because
return;

TOP = TOP + 1;

STACK[TOP] = x; }

~~void Push(int x)~~

~~{ TOP = TOP + 1; → may cause array
STACK[TOP] = x; } out of bounds.~~

array size is

fixed, so check before adding

→ int Pop()

{ if (TOP == -1)
return INT_MIN; }

} Check if empty

int temp = STACK[TOP];

TOP--; → reduce stack element size/index

return temp; }

→ #define SIZE 10

struct STACK {

int Arr[10];

int TOP; };

void main()

{ struct STACK s1, s2;

s1.TOP = s2.TOP = -1;

Push(&s1, 10); }

void Push(struct STACK* ps,
int x)

{ if (ps->TOP == SIZE - 1)
return;

ps->TOP = ps->TOP + 1;

ps->Arr[ps->TOP] = x; }

Stack Permutation

- * Order of insertion of given elements is fixed
- * What could be possible order of pop (stack permutation)

$$\boxed{n! \text{ permutation} \Rightarrow \frac{2n C_n}{n+1} \text{ Stack permutation}}$$

Infix evaluation : $2+3$: Operator is in between Operands

Prefix evaluation : $+23$: Operator is before Operands

Postfix evaluation : $23+$: Operator is after Operands

eg: infix : $2+3 \times 5$

Postfix : $235 \times +$

* Infix to prefix & postfix Conversion

using stack: "evaluation"

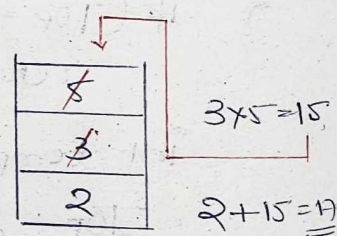
Operator $\Rightarrow +$

(a) pop first element $\Rightarrow A$

(b) pop 2nd element $\Rightarrow B$

$B + A$

$\rightarrow$ Push result to stack & continue evaluation.



eg:2 infix : $2+3 \times 4-6/2$

$\rightarrow$ Postfix = $234 \times + 62 / -$

eg:3 infix = $2+3 \times 5$

$\rightarrow$ Prefix = $+2 \times 35$

eg:4

infix = $(a+b) \times c/d - e/f + g/h$

Postfix : $ab+c \times d / efg$
 $\uparrow \uparrow b / -$

Postfix Evaluation

Postfix: $234 \times + 62 / -$

Scan from left to right.

$23 \overbrace{4}^{\times} + 62 / -$

$2 \overbrace{12}^{+} 62 / -$

$14 \overbrace{62}^{/} -$

$14 \overbrace{3}^{-} \Rightarrow 11$

Whenever you encounter a
operator use it between previous
'2' operands and add result
to stack.

Prefix Evaluation

infix = $2 + 3 \times 5$

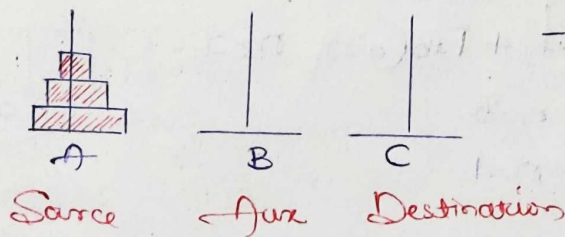
prefix = $+ 2 \times 3 5$

reverse prefix = $5 3 \times 2 +$

Scan left to right & process like
postfix evaluation.

Tower of Hanoi Problem: Given 3 pegs/ and n disc each
of distinct size.

$n=3$



Target: move all disc from
Source peg to
destination peg.

- * We cannot put a large size disc above smaller disc
- * We can move only one disc at a time.

→ void Toy (int n, char source, char dest, char aux)

{ if (n==1)

{ printf ("%c → %c", source, dest);
return; }

Toy (n-1, source, aux, dest);

printf ("%c → %c", source, dest);

Toy (n-1, aux, dest, source); }

function call = $1 + 2 + 2^2 + \dots + 2^{n-1}$ (geometric)

$$= \frac{(2^n - 1)}{2 - 1} = 2^n - 1$$

$$\Rightarrow \underline{\underline{O(2^n)}}$$

moves in Toy = $2^n - 1$

Fibonacci Series : ⁽ⁿ⁾
0 1 2 3 4 ...
1 1 3 5 8 →
_{H(n)}

$$\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2) \quad n \geq 2$$

$$= 0 \quad n=0$$

$$= 1 \quad n=1$$

→ int fib (int n)

{ if (n==0 || n==1) return n;

return fib(n-1) + fib(n-2); }

→ Let $H(n)$ be no. of function calls:

$$H(n) = H(n-1) + H(n-2) + 1$$

$$H(0)=1, H(1)=1$$

→ Let $Q(n)$ be total no of additions in $\text{fib}(n)$

$$Q(n) = Q(n-1) + Q(n-2) + 1$$

$$Q(0) = 0$$

$$Q(1) = 0$$

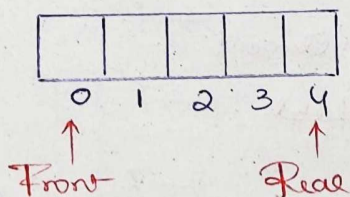
→ Function call invoked before 1st addition takes place in $\text{fib}(n) \Rightarrow n+1$.

Queue

- * Linear data structure
- * First in first out policy. (FIFO).
- * Order of deletion is same as order of insertion.

Insertion : Rear
Deletion : Front

Simple Queue



Initially when Queue is Empty.

$\left\{ \begin{array}{l} \text{Front} = -1 \\ \text{Rear} = -1 \end{array} \right\}$

define SIZE 6

int Queue [SIZE];

int Rear, Front;

Front ~~Rear~~: index from which an element can be deleted.

~~Front~~ : Rear: Index of most recently added element.

When we insert 1st element

$$F = R = 0$$

10	20	30	40		
0	1	2	3	4	5

F ↑ ↑ R

Inserting element
↓

void EnQueue(int x)

{ if (Rear == SIZE-1)
return; } // if queue is full

else if (Front == -1)
{ Rear = Front = 0; Queue[Rear] = x; } // if queue is empty

else { Rear++; Queue[Rear] = x; }

	F	R
Initially	-1	-1
Insert(10)	0	0
Insert(20)	0	1
Insert(30)	0	2
Insert(40)	0	3
delete	1	3

deletion
↓

int DeQueue()
{ if (Front == -1)
return INT_MIN; } // If empty

int temp;

else if (Rear == Front)

{ temp = Queue[Front];
Front = Rear = -1;
return temp; } // Only 1 element available

else { temp = Queue[Front];
Front++;
return temp; }

Queue is Full?

$$\text{Front} == 0 \ \& \ \text{Rear} == \text{SIZE}-1$$

→ Problem with linear queue: →

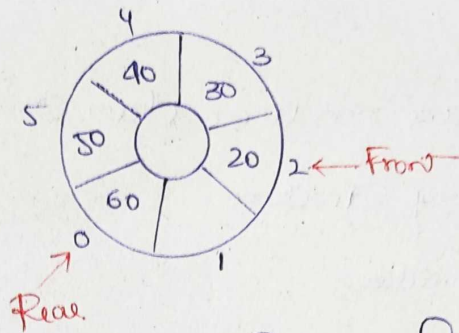
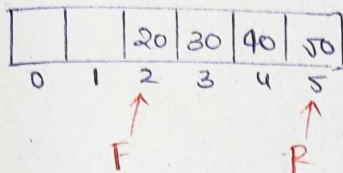
				40	50
0	1	2	3	4	5

F ↑ ↑ R

Space available

Overflow!

Circular Queue



EnQueue(60) $\rightarrow$ Rear = 0

Queue[Rear] = x

Queue is full if

- (i) $F = 0$ & $R = \text{SIZE} - 1$
- (ii) $F = R + 1$

void CQ-Enque(int x)

```
{ if (Front == (Rear + 1) % SIZE)
    return;
```

$F = (R + 1) \% \text{SIZE}$

```
else if (Rear == SIZE - 1) Rear = 0;
```

```
else if (Front == -1) Front = Rear = 0;
```

```
else { Rear++; Queue[Rear] = x; }
```

```
}
```

int CQ-Delete()

```
{ int temp;
```

```
if (Front == -1) return INT_MIN;
```

```
if (Front == Rear) { temp = Queue[Front];
                    Front = Rear - 1; return temp; }
```

```
else if (Front == SIZE - 1) { temp = Queue[Front];
                             Front = 0; return temp; }
```

```
else { temp = Queue[Front];
      Front++; return temp; }
```

```
}
```


TREE

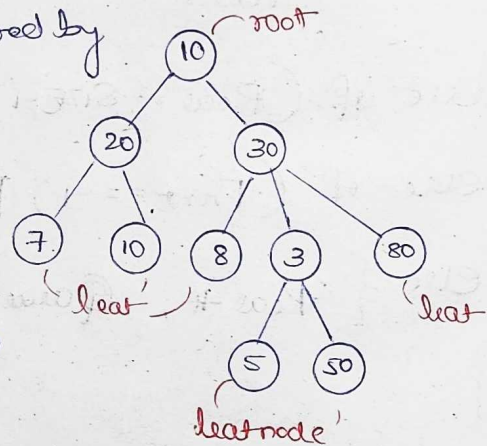
28/11/23

- * Trees are non linear data structure
- * Directory structure
- * Organisation

- Applications of Tree :
1. HTML/tmL
 2. Parse tree
 3. Binary Search Tree
 4. Binary heap
 5. B-tree, B⁺ Tree.

Tree Terminologies :

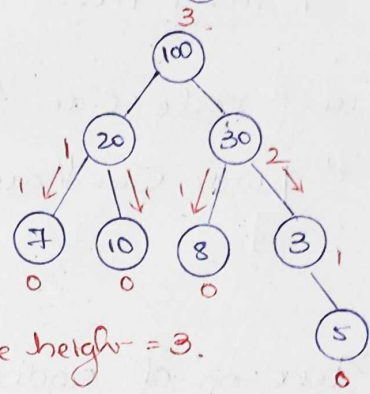
- ① Node : Each element is represented by a node.
- ② Child : Node below another node is called as child node. eg: 7 is child of 20.
- ③ Parent : Node which is directly above another node is called parent node. 20 is parent of 7.
- ④ Root Node : Distinguishable node. Only node without any parent. Eg: 10 is the root.
- ⑤ Leaf node : A node without any child. eg: 5, 50...
- ⑥ Internal node : Node with atleast 1 child is called internal node.
- ⑦ Degree of a node : In tree, degree of a node represents the no of child of a node.



⑧ Height of a node: Height of a node "x" is the length of path from node x to the farthest leaf node.

* Height of leaf node = 0.

* Height of tree = height of root node.

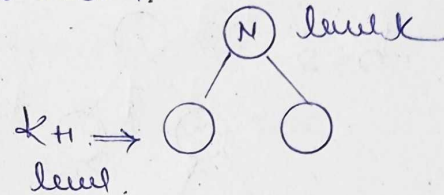


⑨ Level/Depth of a node: Level of a node

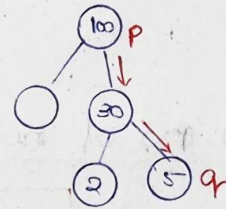
Tree height = 3.

is the length of path from root node to node x.

* Level of root node = 0.



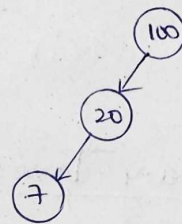
⑩ Ancessor of a node: If there is a path from node p to node q, then all the nodes in the path other than q are called as ancestor of q. Eg: 30, 100 are ancestor of 50.



* Parent of a node is an ancestor. and Parent of some ancestor is also an ancestor.

⑪ Descendant of a node: It is either a child of the node or child of some descendant of a node.

* 7 is a descendant for 20 and 100.



⑫ Sibling: Nodes with same parent are called sibling.

* Root node cannot have a sibling.

⑬ Generation: All nodes at a particular level belong to same generation.

⑭ Size of a node: The no of descendant of a node (including the node itself) is called as size of the node.

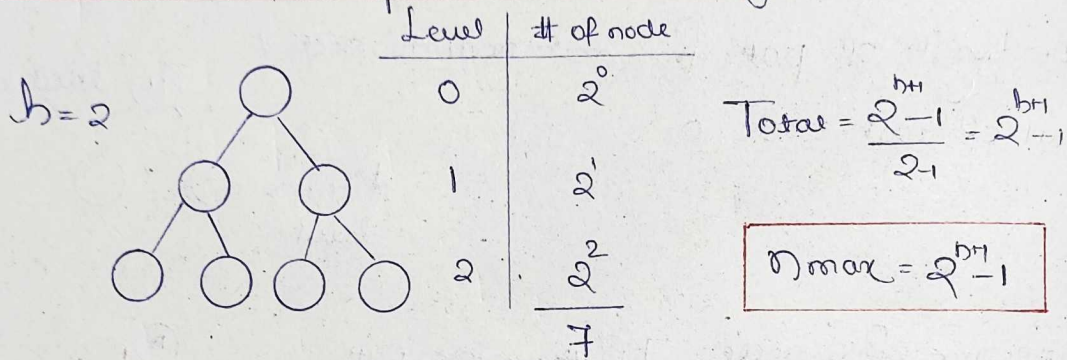
Binary Tree

Every node can have at most 2-child:

Node can have $\Rightarrow$

- 0 $\rightarrow$ Child (leaf node)
- 1 $\rightarrow$ Child
- 2 $\rightarrow$ Child [Internal node]

Max-no of nodes possible in a binary tree of height h :



Min no of nodes in a binary tree of height h :

$$n_{min} = h + 1$$

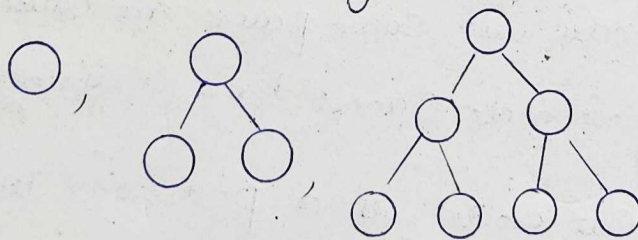
Structure of node:

Struct Node

```

{
    Struct Node * left;
    Int data;
    Struct Node * right;
};
    
```

Full Binary Tree: Tree in which every node have either 0 or 2 nodes is called full binary tree.

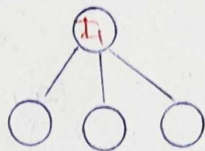


2-ary Tree: A tree in which every internal node has exactly 2 children.

$\rightarrow$ Total nodes = (no. of internal nodes) $\times$ 2 + 1 (root node) = $I \times 2 + 1$

$\uparrow$ internal node
 $\uparrow$ root
 $\downarrow$ children

3-ary tree: A tree in which every internal node has exactly 3 children.



→ Total nodes = $1 \times 3 + 1$

$\swarrow$ no. of internal nodes $\searrow$ root
 $\searrow$ children

K-ary Tree: A tree in which every internal node has k children.

Let I be the no. of internal nodes in a k -ary tree:

Total nodes = $I \times k + 1$ (root)

$$n = k \cdot I + 1$$

leaf nodes + # internal nodes = $kI + 1$

$$L + I = k \cdot I + 1$$

$$L = k \cdot I - I + 1$$

$$L = (k-1)I + 1$$

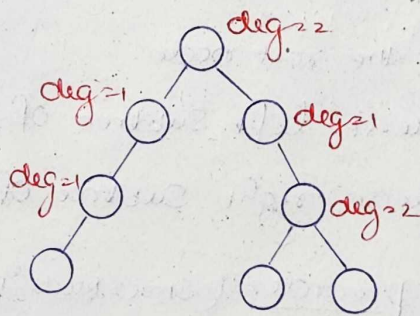
$$I = \frac{L-1}{k-1}$$

Q1) A binary tree with

2 nodes of degree 2

3 nodes of degree 1

Find no. of leaf nodes

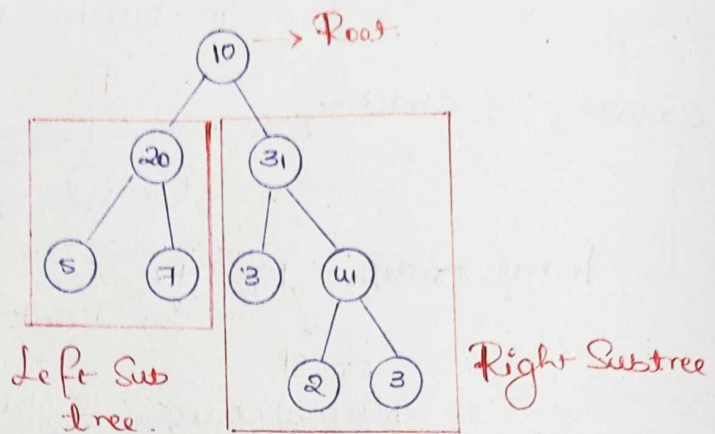


→ $n = 2 \times 2 + 3 \times 1 + 1$
 $= 8$ leaf nodes.

Tree Traversal

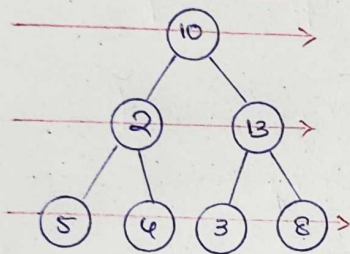
1. Root
2. Left Subtree
3. Right Subtree

* Tree can be traversed by Recursion



Tree Traversal

Level Order



⇒ 10, 2, 13, 5, 4, 3, 8

Depth order

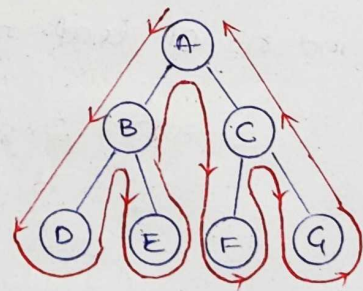
- ① Root, LT, RT
- ② LT, Root, RT
- ③ LT, RT, Root

Main 3 traversals

Pre-order Traversal

1. Visit the root node
2. Traverse left subtree of root node in pre-order.
3. Traverse right subtree of root node in pre-order.

```
void preorder (struct Node *ptr)
{
    if (ptr != NULL)
    {
        printf("%d", ptr->data);
        preorder (ptr->left);
        preorder (ptr->right);
    }
}
```

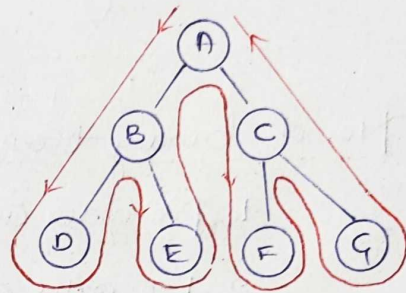


Traversal Order : ABDECFG

In-order Traversal

1. Traverse Lt of root node in In-order.
2. Print/visit root node
3. Traverse Rt of root node in In-order.

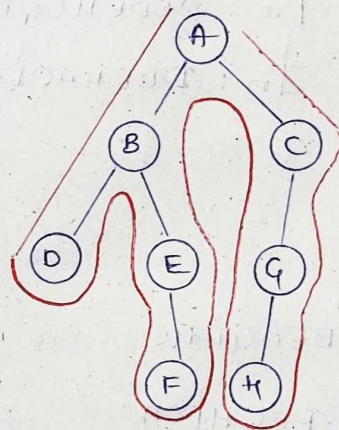
```
void Inorder (Struct Node *ptr)
{
    if (ptr != NULL)
    {
        Inorder (ptr->left);
        printf ("%d", ptr->data);
        Inorder (ptr->right);
    }
}
```



⇒ DBEAFCG

Post-Order Traversal : Lt, Rt, Root.

```
void PostOrder (Struct Node *ptr)
{
    if (ptr)
    {
        PostOrder (ptr->left);
        PostOrder (ptr->right);
        printf ("%d", ptr->data);
    }
}
```



⇒ DFEB HGC A
Lt Rt Root.

→ No of unlabelled binary trees with n nodes
(Shape/Structure) = $\frac{2^n C_n}{n+1}$



⇒ No of labelled binary trees with n nodes = $\frac{2^n C_n}{n+1} \times n!$

With a pre-order given (or length), no of binary trees possible = $\frac{2^n C_n}{n+1}$

With a given postorder (n length), no. of binary tree = $\frac{2^n n!}{n+1}$

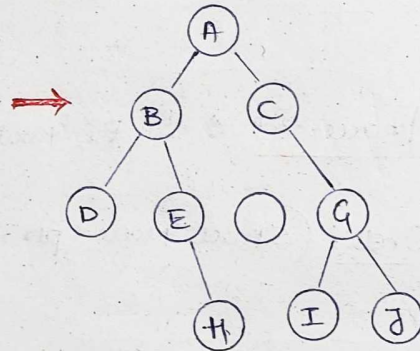
No. of binary trees with a given traversal (Pre/In/Post) of n length = $\frac{2^n n!}{n+1}$

No. of binary tree with:

1. Pre-order and Post-order $\Rightarrow$ many
2. Pre-order and In-order $\Rightarrow 1$
3. Post-order and In-order $\Rightarrow 1$

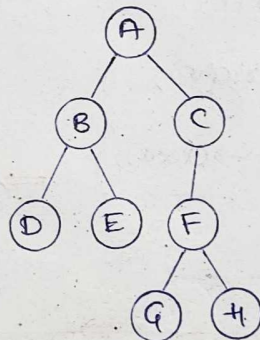
Construct Binary Tree

Pre: ABDEHCGIJ
In: DBEHACIEGJ

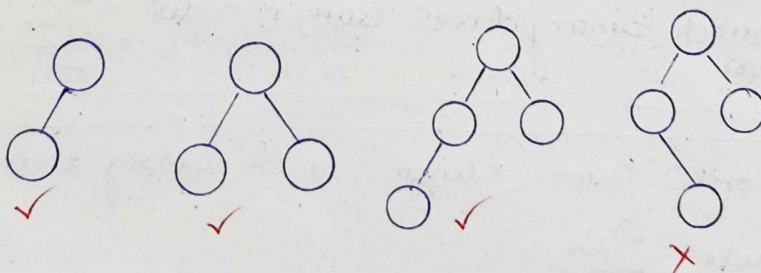


In: DBEAGFHC

Post: DEBGFHCA



Complete Binary Tree: A tree in which is full upto second last level and nodes at last level are filled from left to right.



* Max no of nodes in a CBT of height $h = 2^{h+1} - 1$

29/11/23

* Min no of nodes in CBT of height $h = 2^h$

$$2^h \leq n \leq 2^{h+1} - 1$$

$$2^h \leq n$$

$$\log 2^h \leq \log n$$

$$h \log 2 \leq \log n$$

$$h \leq \frac{\log n}{\log 2}$$

$$h \leq \log_2 n$$

$$\log_2(n+1) - 1 \leq h \leq \log_2 n$$

$$n \leq 2^{h+1} - 1$$

$$n+1 \leq 2^{h+1}$$

$$\log(n+1) \leq (h+1) \cdot \log 2$$

$$\frac{\log(n+1)}{\log 2} \leq h+1$$

$$h+1 \geq \log_2(n+1)$$

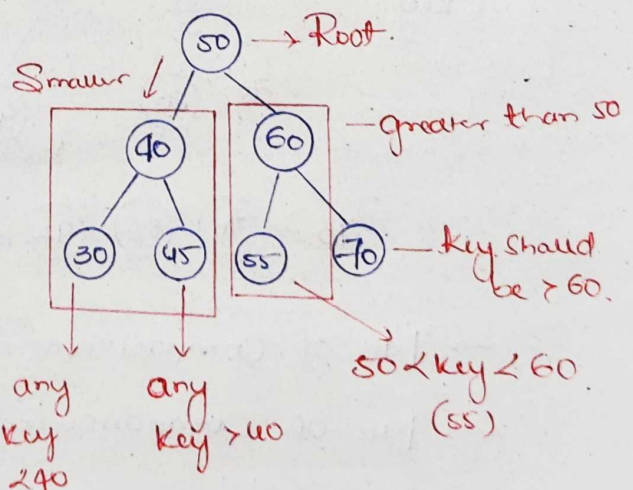
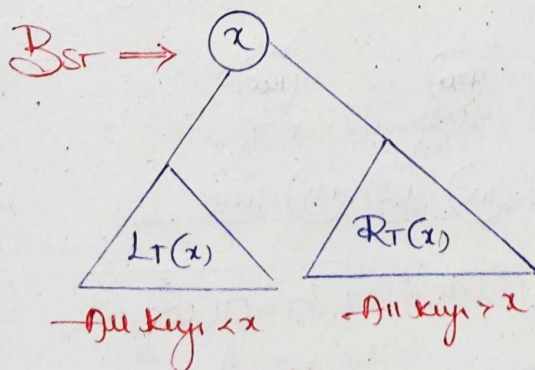
$$h \geq \log_2(n+1) - 1$$

Binary Search Tree

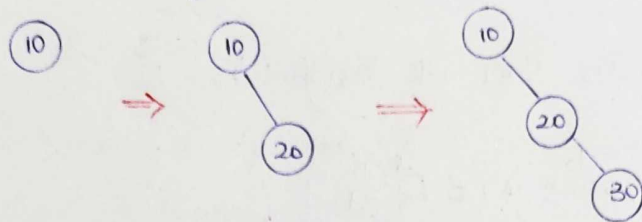
A BST is a binary tree in which every node satisfies the Property : *

- All the keys in the left subtree of a node (x) are smaller than x .

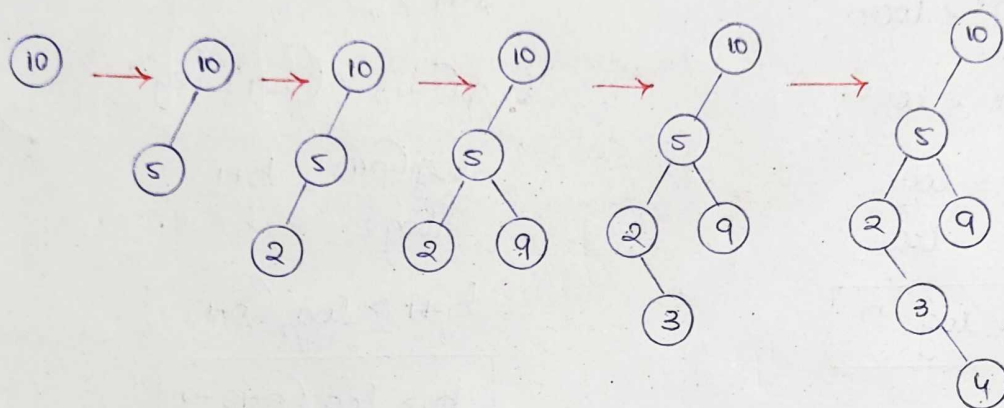
- All the keys in the right subtree of node (x) are greater than x .



① Construct BST by inserting keys 10, 20, 30 in the same order.



② Construct BST by inserting following keys in order: 10, 5, 2, 9, 3, 4



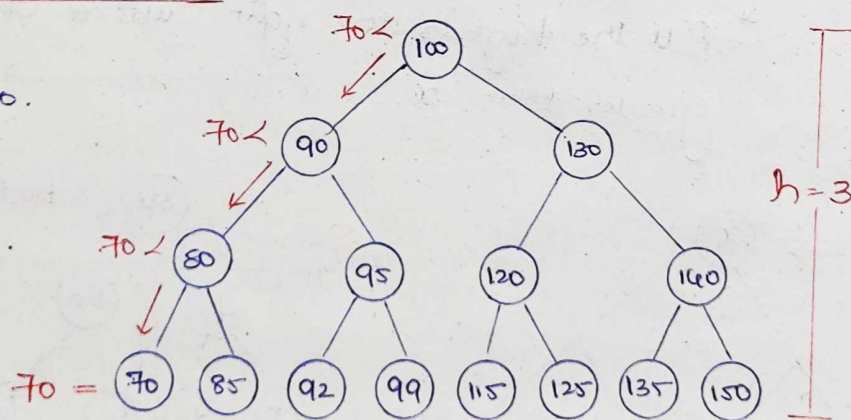
* No of BSTs, when the insertion order of keys are given = 1

* No of BSTs with n keys = $\frac{2^n n!}{n+1}$

* The inorder traversal of a BST is always increasing order of keys

Search in a BST

$k = 70$
(key)



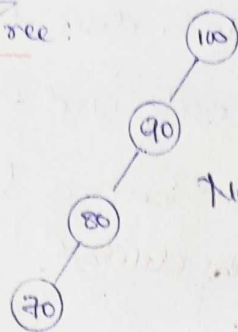
→ No of comparisons = 4

→ No of comparisons = $O(h)$

$h = O(\log_2 n)$

IF tree is balanced.

Squashed Tree:



No of comparisons in worst case = $O(n)$

$$h = (n-1)$$

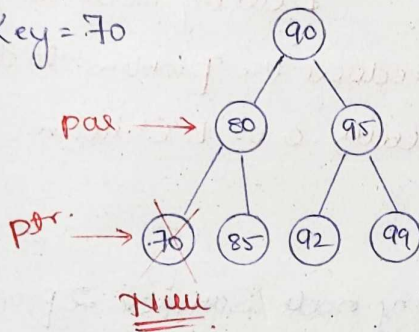
Deletion:

Case 1 - Deletion of a node having 0-child (leaf node)

Case 2 - Deletion of a node having 1-child

Case 3 - Deletion of a node having 2-child.

Key = 70



We need to identify the parent pointer (of node to be deleted) which is pointing to the node to be deleted.



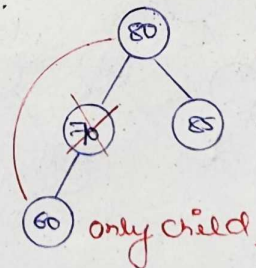
"Make this pointer Null"

if ($ptr \rightarrow data < par \rightarrow data$)
 { $par \rightarrow left = null$;
 $free(ptr)$; }

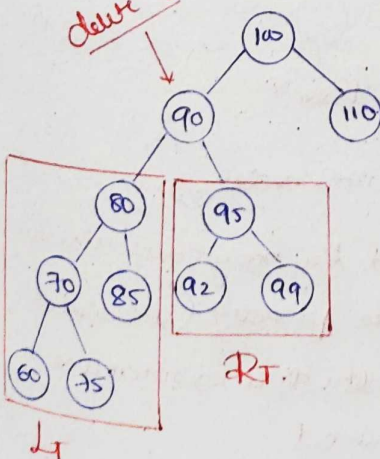
else { $par \rightarrow right = null$;
 $free(ptr)$; }

Deletion of node having 1 child

Key = 70



delete 90

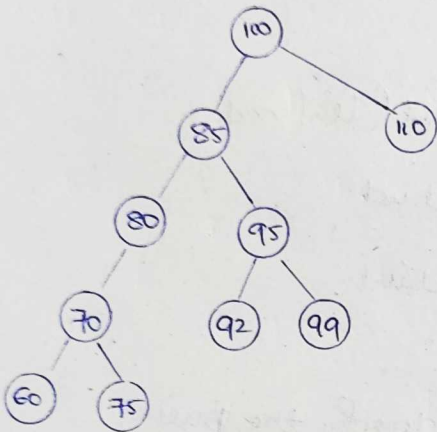


1st way: Replace the node (key) to be deleted with the maximum key in the left subtree of node and then perform deletion.

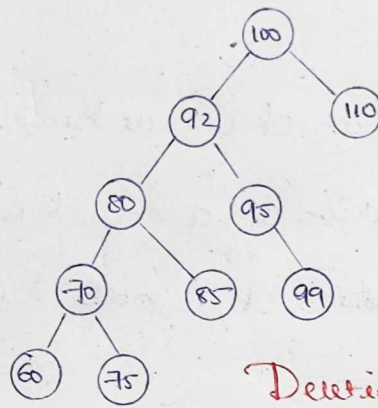
$90 \leftrightarrow 85$ and since it becomes leaf node, it can be deleted

2nd way: Replace the node (key) to be deleted with the minimum key in the right sub tree of the node and then perform deletion.
 $90 \Rightarrow 92$ and it becomes leaf node, thus can be deleted.

By 1st way.



By 2nd way.

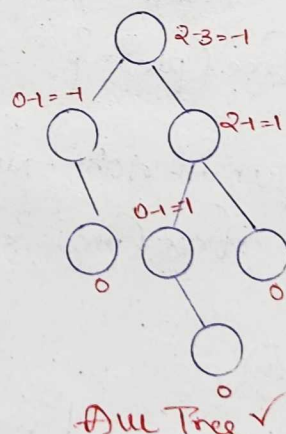
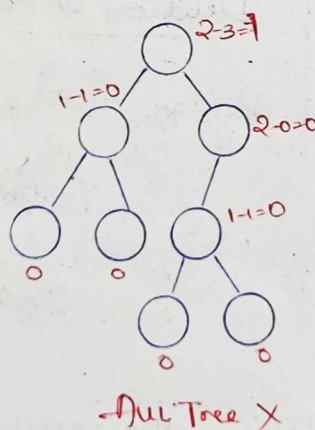
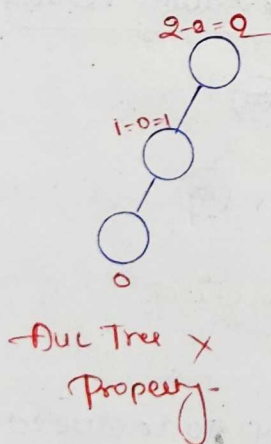


Deleting node with 2 Children reduces the problem to deleting a node with 0 or 1 Children.

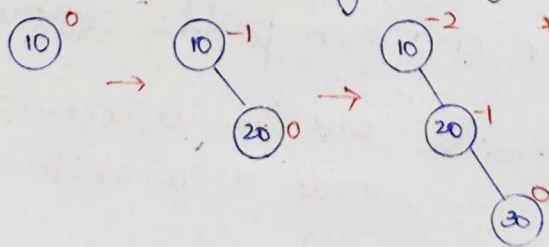
AVL Tree

$\Rightarrow$ AVL Tree is a tree in which every node satisfies 2 properties:

- ① BST Property
- ② The balancing factor of a node can be either 0, -1, or +1.



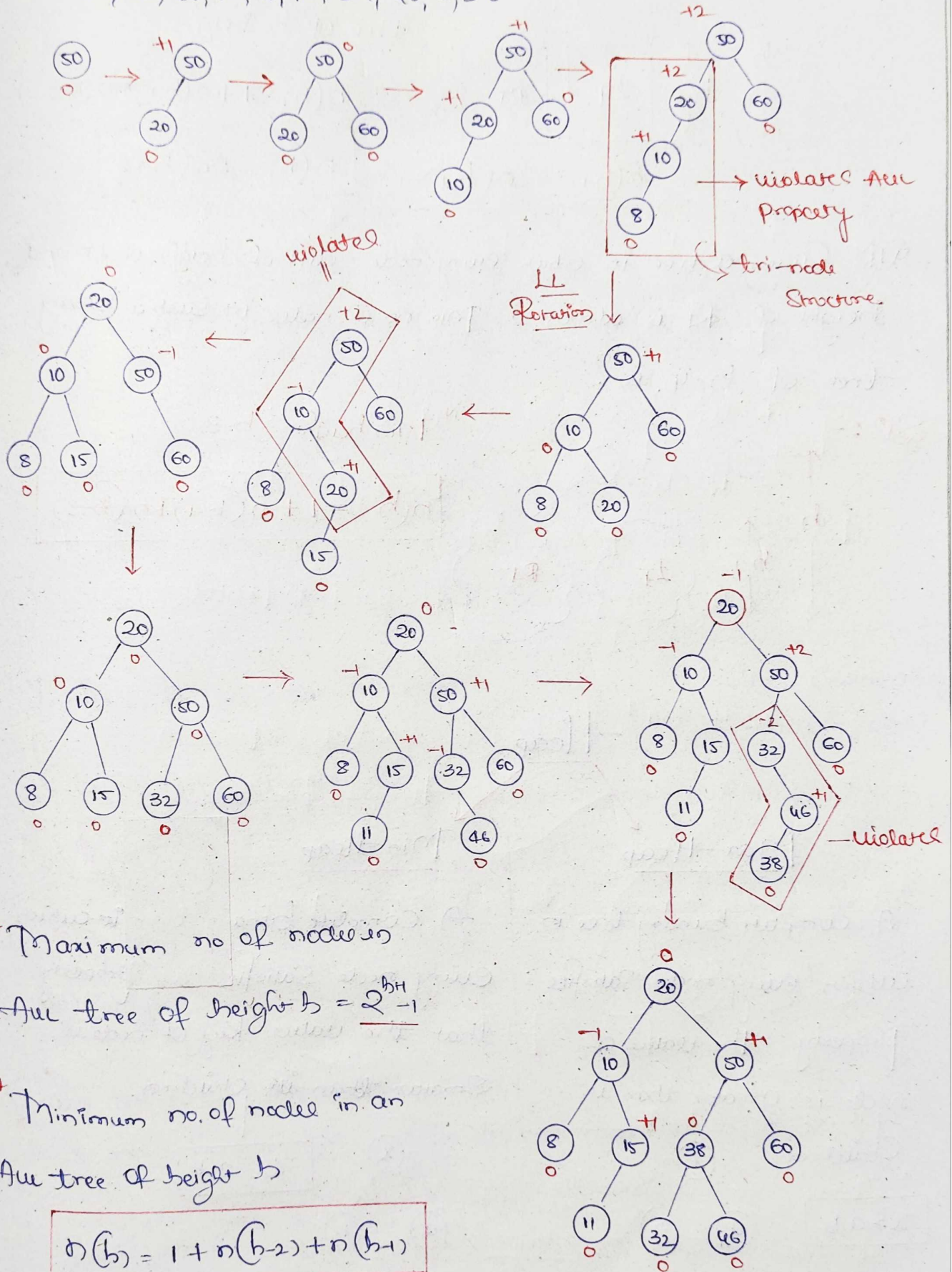
$\Rightarrow$ Construct an AVL tree by inserting 10, 20, 30 in same order.



* Insertion of key may cause violation in AVL tree property (unbalanced). To balance the tree $\Rightarrow$ rotations are required.

→ Construct AVL tree by inserting keys in given order

50, 20, 60, 10, 8, 15, 32, 46, 11, 38



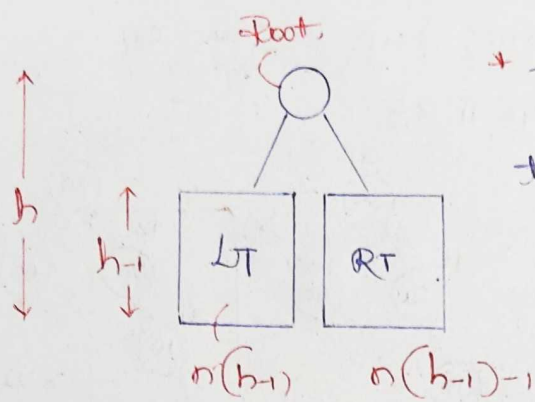
* Maximum no of nodes in

AVL tree of height $h = 2^h - 1$

* Minimum no. of nodes in an

AVL tree of height h

$$n(h) = 1 + n(h-2) + n(h-1)$$



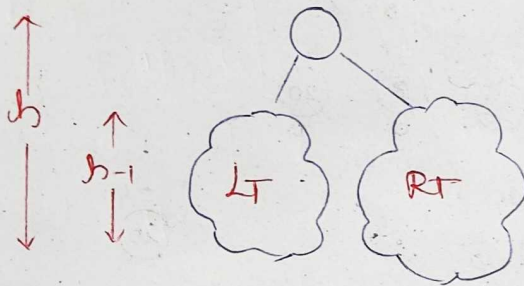
* Min no of nodes in such given tree of h height

$$n(h) = 1 + n(h-1) + n(h-1) - 1$$

$$n(h) = 2n(h-1)$$

Q1. Given a tree in which every node = diff of height of LT and height of RT is at most 2. Min no of nodes in such a binary tree of $h=4$. i.e.?

Solⁿ :-



* Min height = $h-3$.

$$n(h) = 1 + n(h-1) + n(h-3)$$

Heap

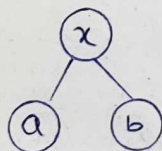
Max-Heap

A complete binary tree in which every node satisfies

Property: The value of node is greater than its

Children

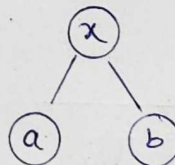
$$x > a, b$$



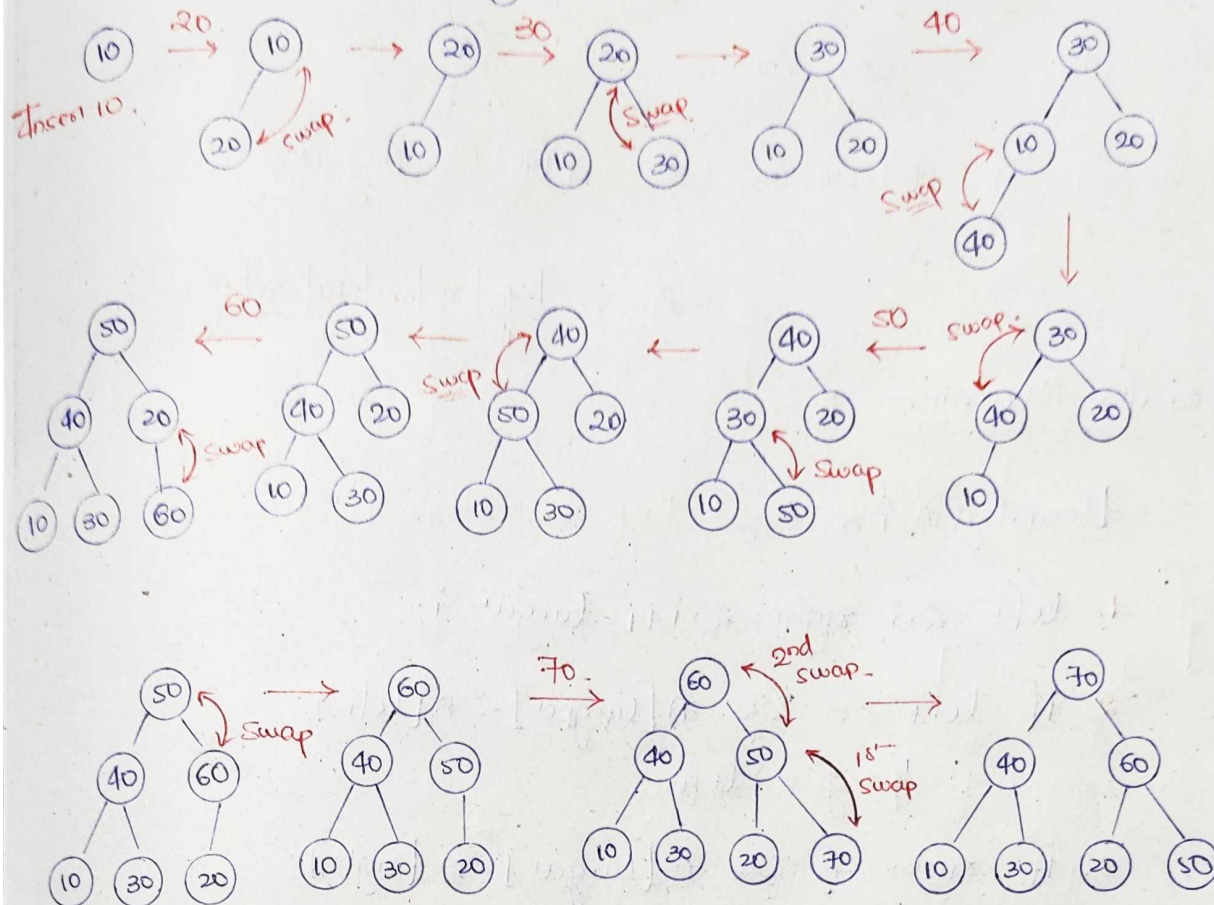
Min-Heap

A complete binary tree in which every node satisfies the property that the value/key of node is smaller than its children.

$$x < a, b$$



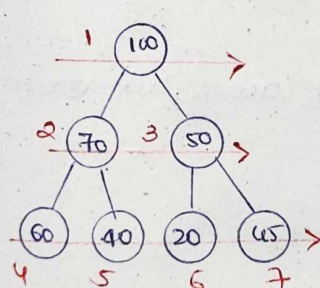
→ Construct Max heap by inserting keys 10, 20, 30, 40, 50, 60, 70 in some order.



* Insertion in heap with n nodes = $O(\log_2 n)$

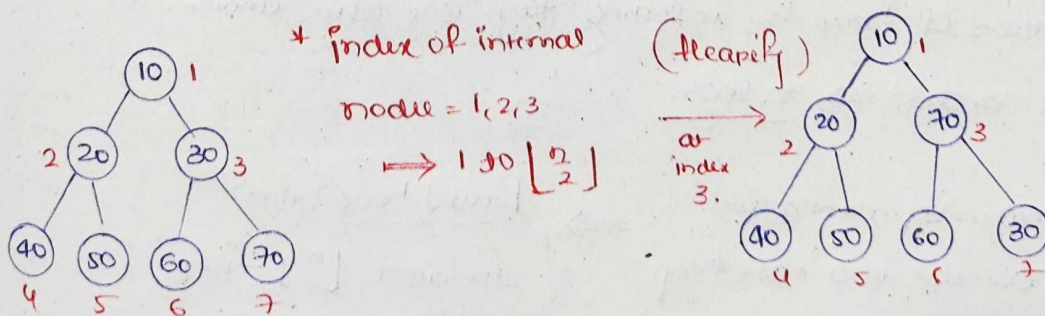
(Heap Construction takes $O(\log n)$)

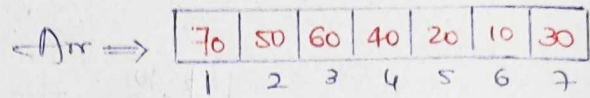
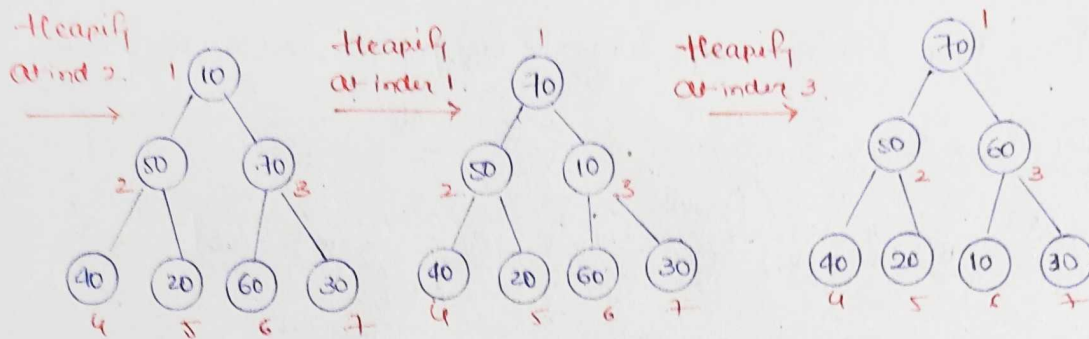
CBT $\Rightarrow$ Array Representation



100	70	50	60	40	20	45
1	2	3	4	5	6	7

Given an array representation of a: 10, 20, 30, 40, 50, 60, 70.
Is it max heap? — NO, Convert into max heap.





Heapify Algorithm

Heapify(A, i, n)

1. left = $2 \times i$, right = $2 \times i + 1$, largest = i ;
2. if left $\leq n$ && $A[\text{largest}] < A[\text{left}]$:
largest = left;
3. if right $\leq n$ && $A[\text{largest}] < A[\text{right}]$:
largest = right;
4. if $i \neq \text{largest}$ { swap $(A[i], A[\text{largest}])$
Heapify $(A, \text{largest}, n)$; }

* With array representation we can get parent along with existing node.

$$\left. \begin{array}{l} \text{Node} \Rightarrow i \\ \text{Parent} \Rightarrow \left\lfloor \frac{i}{2} \right\rfloor \end{array} \right\} \text{index.}$$

* No of leaf nodes in a heap with n nodes = $\left\lceil \frac{n}{2} \right\rceil$

* Construction of heap by inserting keys one after another in a given order $\Rightarrow \underline{n \log n}$

* Given array representation of CBT, convert into max-heap $\Rightarrow$ Build-heap(A, n)

$$\left\{ \begin{array}{l} \text{for } (i = \left\lfloor \frac{n}{2} \right\rfloor : i \geq 1; i--) \\ \quad \text{Heapify}(A, i, n); \end{array} \right\}$$

Level	# Comp. for each node	# nodes at each level
0	$2(h)$	2^0
1	$2(h-1)$	2^1
2	$2(h-2)$	2^2
...	...	...

$$\frac{S}{2} = 2^{h+1} - 2 - h$$

← Summation of series

$$n_{\max} = 2^{h+1} - 1$$

$$\Rightarrow \frac{S}{2} = n+1 - 2 - \log_2 n$$

$$\Rightarrow \frac{S}{2} = n - 1 - \log_2 n$$

$$\Rightarrow S = 2n - 2 - 2\log_2 n$$

$$S = O(n) \rightarrow \text{Build heap Complexity.}$$

Given Max Heap:

1	2	3	4	5	6	7
100	70	80	30	40	28	60

* Find "max-element": Arr[1] $\rightarrow O(1)$

* Find "min-element": Can be some leaf node

$$\Rightarrow \text{no. of leaf nodes} = \lceil \frac{n}{2} \rceil \Rightarrow O(n)$$

$$\Rightarrow \text{no. of comparisons} = (n-1)$$

Min heap Find min: $O(1)$
Find max: $O(n)$

Extract max: get the max element from heap and delete it from heap. (In "max heap")

$$\Downarrow$$

$$O(\log_2 n) = \text{Find max } O(1) + \text{Deletion } O(\log_2 n)$$

Max-heap

* Deletion: $O(\log_2 n)$

* Insertion: $O(\log_2 n)$

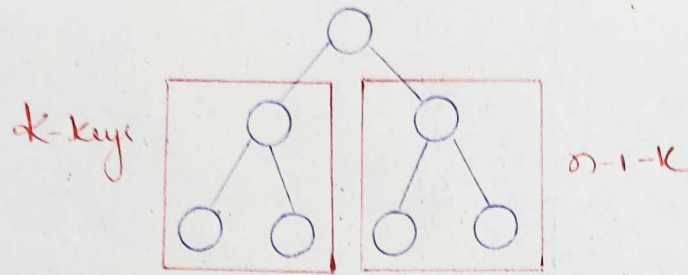
* Search: $O(n)$

* Find_max: $O(1)$

* Find_min: $O(n)$

* Extract_max: $O(\log_2 n)$

→ $T(n)$: no of max-heap cuts in distinct keys



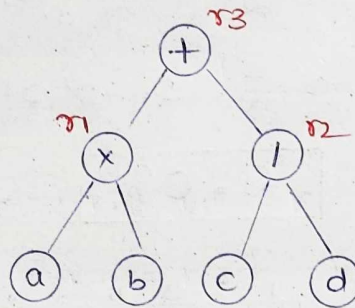
$$T(n) = 1 \times {}^{n-1}C_k \times T(k) \times T(n-k-1)$$

Select any 'k' keys from (n-1) for LT

Expression Tree

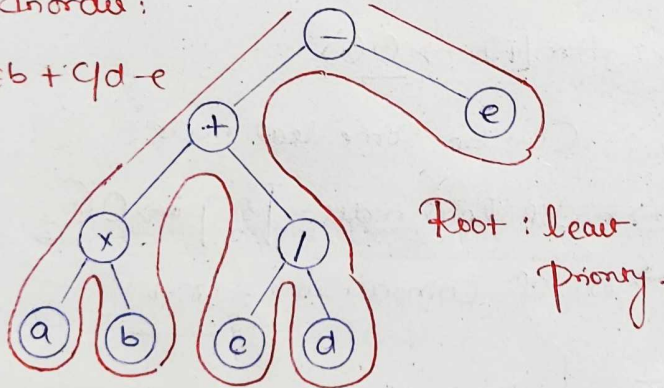
Infix: $\frac{a \times b}{r_1} + \frac{c/d}{r_2} - e$

$(r_1 + r_2) - e$
 $r_3 - e$



Inorder:

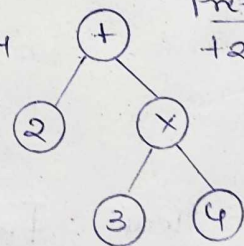
$a \times b + c/d - e$



Prefix to Expression tree

Infix: $2 + 3 \times 4$

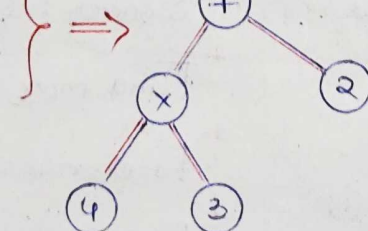
Prefix: $+ 2 \times 3 4$



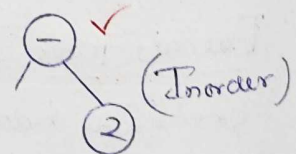
Post-fix to Expression tree

Infix: $4 \times 3 + 2$

Postfix: $4 3 \times 2 +$



Unary operator: - 2



* Unary operator > Binary operator

GRAPHS

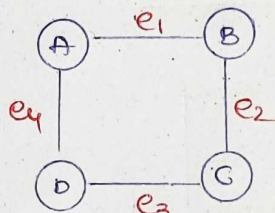
- * A Graph is a non linear data structure.
- * It consist of vertices and edges.
- * The vertices are sometimes also referred to as nodes, and edges are lines or arcs that connect any two nodes in the graph.

$G(V, E)$

Set of vertices Set of edges

$V = \{A, B, C, D\}$

$E = \{e_1, e_2, e_3, e_4\}$



Graph Traversal

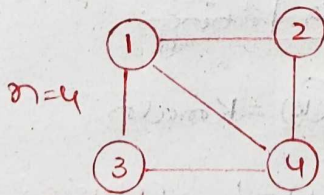
1. Breadth first search
2. Depth first search

Graph Representation

1. Adjacency matrix
2. Adjacency list

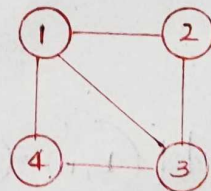
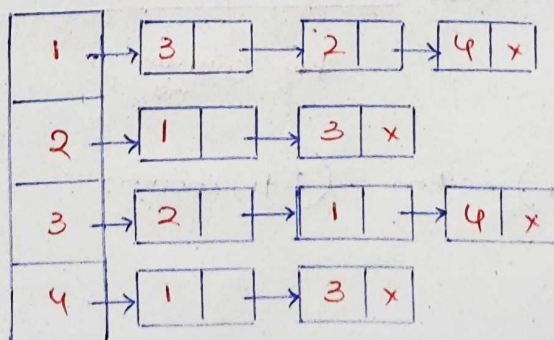
Adjacency Matrix: A $n \times n$ matrix in which every entry is either 0 or 1

$$\left\{ \begin{array}{l} A_{ij} = 1, \text{ if node } i \text{ is adjacent to node } j \\ = 0, \text{ otherwise} \end{array} \right\}$$



$$A = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix} \end{matrix}$$

Adjacency List

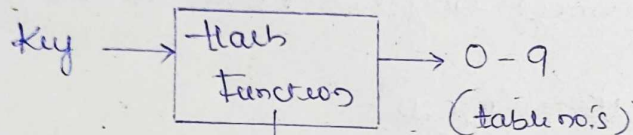


Hashing

* Hashing is a technique of mapping a large chunk of data into small table using a hashing function.

→ $m=10$

→ Keys = 13, 22, 15, 78, 86, 91, 107.



$$h(\text{key}) = (\text{key}) \bmod m$$

0	
1	91
2	22
3	13
4	
5	15
6	86
7	107
8	78
9	

$$h(13) = 13 \bmod 10 = 3$$

$$h(22) = 22 \bmod 10 = 2$$

$$h(15) = 15 \bmod 10 = 5$$

$$h(78) = 78 \bmod 10 = 8$$

$$h(86) = 86 \bmod 10 = 6$$

$$h(91) = 91 \bmod 10 = 1$$

$$h(107) = 107 \bmod 10 = 7$$

$$\left. \begin{array}{l} h(k_1) \rightarrow 1 \\ h(k_2) \rightarrow 1 \end{array} \right\} \text{Collision.}$$

Collision Resolution Technique.

- ① Linear Probing
- ② Quadratic Probing
- ③ Double hashing
- ④ Separate Chaining.

Linear Probing

$$\text{Let } h(k) = k \bmod m$$

$$h(k_1) = L \quad \left. \vphantom{h(k_1)} \right\} \text{collision occur.}$$

$$H(k, i) = (h(k) + i) \bmod m$$

key collision
 no.

$$\Rightarrow H(k, 1) = (h(k) + 1) \bmod m = (L + 1) \bmod m$$

Keys = 31, 26, 43, 27, 34, 46, 14, 58, 13.

$m = 12$

(i) $h(31) = 31 \bmod 12 = 7$

(ii) $h(26) = 26 \bmod 12 = 2$

(iii) $h(43) = 43 \bmod 12 = 7$ (Collision)
 $i = 1$

$h(k, i) = (h(k) + i) \bmod m$

$h(43, 1) = (h(43) + 1) \bmod 12$
 $= (7 + 1) \bmod 12 = 8$

(iv) $h(27) = 27 \bmod 12 = 3$

(v) $h(34) = 34 \bmod 12 = 10$

(vi) $h(46) = 46 \bmod 12 = 10$ (Collision)

$h(46, 1) = (h(46) + 1) \bmod 12 = 11$

(vii) $h(14) = 14 \bmod 12 = 2$ - Collision

$h(14, 1) = (2 + 1) \bmod 12 = 3$ - Collision

$h(14, 2) = (3 + 1) \bmod 12 = 4$

(viii) $h(58) = 58 \bmod 12 = 10$

Collision

$h(58, 1) = (10 + 1) \bmod 12$

$= 11$ - Collision

$h(58, 2) = (11 + 1) \bmod 12$

$= 0$

(ix) $h(13) = 13 \bmod 12 = 1$

0	58
1	13
2	26
3	27
4	14
5	
6	
7	31
8	43
9	
10	34
11	46

* Primary Clustering Problem occurs when different keys collide to form one big group.

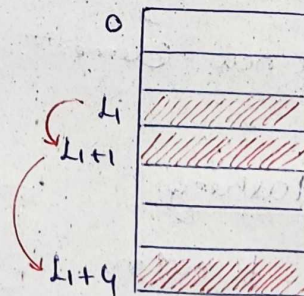
Quadratic Probing:

$h(k) = k \bmod m = L_1$ - Collision

$h(k, i) = (h(k) + i^2) \bmod m$

$h(k, 1) = (h(k) + 1^2) \bmod m$

$h(k, 2) = (h(k) + 2^2) \bmod m$



Keys: 24, 17, 32, 2, 13, 50, 30, 61

$$m=11$$

$$h(k) = k \bmod m$$

$$(i) h(24) = 24 \bmod 11 = 2$$

$$(ii) h(17) = 17 \bmod 11 = 6$$

$$(iii) h(32) = 32 \bmod 11 = 10$$

$$(iv) h(2) = 2 \bmod 11 = 2$$

Collision

$$h(2,1) = (2 + 1^2) \bmod 11 = 3$$

$$(v) h(13) = 13 \bmod 11 = 2$$

Collision

$$h(13,1) = (2 + 1^2) \bmod 11 = 3$$

Collision

$$h(13,2) = (2 + 2^2) \bmod 11 = 6$$

Collision

$$h(13,3) = (2 + 3^2) \bmod 11 = 0$$

✓

$$(vi) h(50) = 50 \bmod 11 = 6$$

Collision

$$h(50,1) = (6 + 1^2) \bmod 11 = 7$$

$$(vii) h(30) = 30 \bmod 11 = 8$$

$$(viii) h(61) = 61 \bmod 11 = 6$$

Collision

$$h(61,1) = (6 + 1^2) \bmod 11 = 7$$

Collision

$$h(61,2) = (6 + 2^2) \bmod 11 = 10$$

Collision

$$h(61,3) = (6 + 3^2) \bmod 11 = 4$$

✓

0	13
1	
2	24
3	2
4	61
5	
6	17
7	50
8	30
9	
10	32

Secondary Clustering Problem

Keys that are hashed to same memory location always follow same resolution path because of which we are not able to utilize the table size efficiently.

In spite of having free slots, we are not able to provide a free slot to a new element.

Double Hashing

$$h(k) = k \bmod m$$

Collision occurs

$$h(k,i) = (h(k) + i \cdot h(k)) \bmod m$$

Primary hash function

Secondary hash function

Q3) Keys: 13, 17, 21, 2, 57, 28, 30, 27.

$$m=11.$$

$$h(x) = x \bmod 11$$

$$h'(x) = 7 - (x \bmod 7)$$

$$(i) h(13) = 13 \bmod 11 = 2$$

$$(ii) h(17) = 17 \bmod 11 = 6$$

$$(iii) h(21) = 21 \bmod 11 = 10$$

$$(iv) h(2) = 2 \bmod 11 = 2$$

Collision.

$$H(2,1) = (h(2) + 1 \cdot h'(2)) \bmod 11$$

$$h'(2) = 7 - 2 \bmod 7 = 5$$

$$H(2,1) = (2 + 1 \cdot 5) \bmod 11 = 7.$$

$$(v) h(57) = 57 \bmod 11 = 2$$

Collision.

$$H(57,1) = (h(57) + 1 \cdot h'(57)) \bmod 11$$

$$= (2 + 6) \bmod 11 = 8.$$

$$(vi) h(28) = 28 \bmod 11 = 6$$

Collision.

$$H(28,1) = (h(28) + 1 \cdot h'(28)) \bmod 11$$

$$= (6 + 7) \bmod 11 = 2$$

Collision.

$$H(28,2) = (h(28) + 2 \cdot h'(28)) \bmod 11$$

$$= (6 + 2 \cdot 7) \bmod 11 = 9.$$

$$(vii) h(30) = 30 \bmod 11 = 8$$

Collision.

$$H(30,1) = (h(30) + 1 \cdot h'(30)) \bmod 11$$

$$= (8 + 5) \bmod 11 = 2$$

Collision.

$$H(30,2) = (h(30) + 2 \cdot h'(30)) \bmod 11$$

$$= (8 + 2 \cdot 5) \bmod 11 = 7$$

Collision.

$$H(30,3) = (h(30) + 3 \cdot h'(30)) \bmod 11$$

$$= (8 + 3 \cdot 5) \bmod 11 = 1.$$

$$(viii) h(27) = 27 \bmod 11 = 5$$

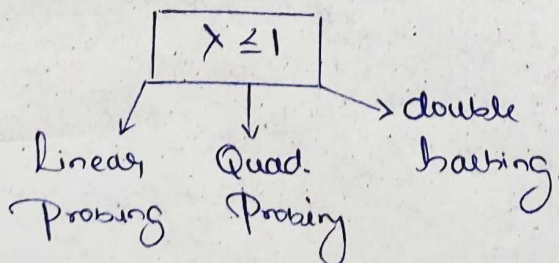
Load Factor (λ)

$$\lambda = \frac{n}{m}$$

$\rightarrow$ no. of keys
 $\rightarrow$ table size.

$$\lambda > 1$$

Seperate Chaining



Separate Chaining:

Keys: 400, 300, 125, 625, 36, 96, 106, 500

$m=10$

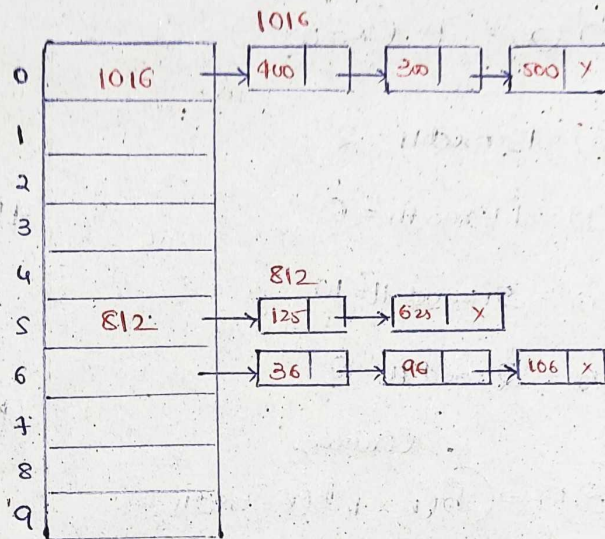
$$h(400) = 400 \bmod 10 = 0$$

$$h(300) = 300 \bmod 10 = 0$$

$$h(125) = 125 \bmod 10 = 5$$

$$h(625) = (625) \bmod 10 = 5$$

$$h(36) = (36) \bmod 10 = 6$$



* Deletion in Separate Chaining is easier.

* In case of linear probing, quadratic probing and double hashing delete operation is expensive. We need to re-hash all the keys.